

PRACTICAL WEEK-5

1. Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

CODE:

```
GNU nano 7.2 producer_consumer_pipe.c *
#include <stdlib.h>
#include <sys/wait.h>
#include <string.h>
#include <time.h>

#define SIZE 1000

int main()
{
    int fd[2];
    pid_t pid;
    char buffer[SIZE];

    clock_t start, end;
    double cpu_time;

    if(pipe(fd) == -1)
    {
        perror("Pipe failed");
        return 1;
    }

    pid = fork();

    if(pid < 0)
    {
        perror("Fork failed");
        return 1;
    }

    if(pid > 0)
    {
        close(fd[0]);

        start = clock();

        for(int i = 0; i < 10000; i++)
```

```

root@SRIVALLI: ~
GNU nano 7.2 producer_consumer_pipe.c *

    start = clock();

    for(int i = 0; i < 10000; i++)
    {
        sprintf(buffer, "Message %d", i);
        write(fd[1], buffer, strlen(buffer) + 1);
    }

    close(fd[1]);

    wait(NULL);

    end = clock();

    cpu_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("\nProducer completed.\n");
    printf("Communication Time: %f seconds\n", cpu_time);
}
else
{
    close(fd[1]);

    while(read(fd[0], buffer, SIZE) > 0)
    {
        // Consumer receives data
    }

    close(fd[0]);

    printf("Consumer completed.\n");
}

return 0;
}

```

RESULT:

```

root@SRIVALLI:~# nano producer_consumer_pipe.c
root@SRIVALLI:~# gcc producer_consumer_pipe.c -o producer_consumer_pipe
root@SRIVALLI:~# ./producer_consumer_pipe
Consumer completed.

Producer completed.
Communication Time: 0.012065 seconds
root@SRIVALLI:~# |

```

REPORT:

In this experiment, a Producer-Consumer communication system is implemented using anonymous pipes in Ubuntu. The parent process acts as the producer and generates data, while the child process acts as the consumer and receives the data through a pipe. The pipe() system call is used to create a communication channel, and fork() is used to create the child process. The producer sends multiple messages through the pipe, and the consumer reads them successfully. The time taken for communication is measured to evaluate the efficiency of data transfer. This experiment helps in understanding Inter-Process Communication (IPC) and shows how related processes can exchange data efficiently in a Linux environment.

2. Develop a program that executes the equivalent of the shell command: `ls -l grep` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main()
{
    int fd[2];

    if(pipe(fd) == -1)
    {
        perror("pipe");
        exit(1);
    }

    pid_t pid1 = fork();

    if(pid1 == 0)
    {
        close(fd[0]);

        dup2(fd[1], STDOUT_FILENO);
        close(fd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp ls");
        exit(1);
    }

    pid_t pid2 = fork();

    if(pid2 == 0)
    {
        close(fd[1]);

        dup2(fd[0], STDIN_FILENO);
```

```

    pid_t pid2 = fork();
    if(pid2 == 0)
    {
        close(fd[1]);

        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);

        execlp("grep", "grep", "^d", NULL);

        perror("execlp grep");
        exit(1);
    }

    close(fd[0]);
    close(fd[1]);

    wait(NULL);
    wait(NULL);

    return 0;
}

```

RESULT:

```

root@SRIVALLI:~# nano ls_grep_pipe.c
root@SRIVALLI:~# gcc ls_grep_pipe.c -o ls_grep_pipe
root@SRIVALLI:~# ./ls_grep_pipe
drwxr-xr-x 3 root root 4096 Aug 10 11:27 OSSP
drwxr-xr-x 6 root root 4096 Aug 20 05:49 SkillWeek1
drwxr-xr-x 2 root root 4096 Aug 22 06:35 test
root@SRIVALLI:~#

```

REPORT:

In this experiment, a program is developed to execute the equivalent of the shell command `ls -l | grep ^d` using system calls in Ubuntu. The `fork()` system call is used to create child processes, `pipe()` creates a communication channel between them, `dup2()` redirects input and output streams, and `exec()` executes the required commands. The first child process runs the `ls -l` command and sends its output through the pipe, while the second child process receives the data from the pipe and executes the `grep` command to filter directory entries. This experiment helps in understanding process creation, inter-process communication, input-output redirection, and command execution in Linux systems.